

```

1 #=====
2 # MIT 6.046J 代码精读 • PS9 - 锦标赛 FPT
3 # 源文件: tournament_fpt.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS9 Problem 9-2: Tournament Edge Reversal -- fixed-parameter algorithm.
7
8 A tournament  $T = (V, E)$ : for every pair  $u, v$  exactly one of  $(u, v), (v, u)$  in  $E$ .
9 A cycle cover is a set  $A$  of edges hitting every directed cycle. TER: does  $T$  have
10 a cycle cover of size  $\leq k$ ?
11
12 Facts used:
13 (a) reversing a MINIMAL cycle cover makes  $T$  acyclic (a transitive tournament).
14 Equivalently: a tournament is acyclic iff it has no directed triangle
15 (3-cycle); so a "cycle cover" = a set of edges hitting every triangle =
16 a triangle hitting set.
17 (b) kernelization: if  $T$  has a triangle-hitting set of size  $\leq k$  then  $|V|$  can be
18 bounded ( $k^2 + 2k$ -vertex kernel) after removing vertices in no triangle.
19 (c) FPT algorithm: branch on a triangle (reverse one of its 3 edges), depth  $k$ 
20  $\rightarrow 3^k * \text{poly}$ , i.e. FPT.
21
22 We implement:
23 * is_acyclic(T) -- via topological ordering;
24 * has_triangle / find_triangle;
25 * min_cycle_cover_bruteforce -- exact optimum (oracle);
26 * fpt_bounded_search(T, k) -- the  $3^k$  branching FPT algorithm;
27 * kernelize(T) -- drop triangle-free vertices.
28 and verify fpt == brute force.
29 """
30 from __future__ import annotations
31
32 import itertools
33 from typing import FrozenSet, List, Optional, Set, Tuple
34
35 Edge = Tuple[int, int]
36
37 class Tournament:
38     def __init__(self, n: int, arcs: Set[Edge]):
39         self.n = n
40         self.arc = arcs # set of directed edges (u,v)
41
42     @staticmethod
43     def from_order_with_flips(order: List[int], flips: Set[Edge] = frozenset()):
44         """Transitive tournament by `order` (i beats j if earlier), then flip the
45         given arcs (used to create cycles)."""
46         n = len(order)
47         pos = {v: i for i, v in enumerate(order)}
48         arcs = set()
49         for u in range(n):
50             for v in range(n):
51                 if u != v and pos[u] < pos[v]:
52                     arcs.add((u, v))
53             for (a, b) in flips:
54                 if (a, b) in arcs:
55                     arcs.discard((a, b))
56                     arcs.add((b, a))
57                 elif (b, a) in arcs:
58                     arcs.discard((b, a))
59                     arcs.add((a, b))
60         return Tournament(n, arcs)
61
62     def reverse(self, edges) -> "Tournament":
63         arcs = set(self.arc)
64         for (u, v) in edges:
65             if (u, v) in arcs:
66                 arcs.discard((u, v))
67                 arcs.add((v, u))
68             elif (v, u) in arcs:
69                 arcs.discard((v, u))
70                 arcs.add((u, v))
71         return Tournament(self.n, arcs)
72
73 def is_acyclic(T: Tournament) -> bool:
74     """A tournament is acyclic iff it admits a topological order (Kahn)."""
75     indeg = [0] * T.n
76     adj = [[] for _ in range(T.n)]
77     for (u, v) in T.arc:
78         adj[u].append(v)
79         indeg[v] += 1
80     from collections import deque
81     q = deque(i for i in range(T.n) if indeg[i] == 0)
82     seen = 0
83     while q:

```

```

86         u = q.popleft()
87         seen += 1
88         for v in adj[u]:
89             indeg[v] -= 1
90             if indeg[v] == 0:
91                 q.append(v)
92     return seen == T.n
93
94
95 def find_triangle(T: Tournament) -> Optional[Tuple[int, int, int]]:
96     """Return a directed 3-cycle (a->b->c->a) if one exists."""
97     arc = T.arc
98     for a, b, c in itertools.permutations(range(T.n), 3):
99         if (a, b) in arc and (b, c) in arc and (c, a) in arc:
100             return (a, b, c)
101     return None
102
103
104 def has_triangle(T: Tournament) -> bool:
105     return find_triangle(T) is not None
106
107
108 def min_cycle_cover_bruteforce(T: Tournament) -> int:
109     """Smallest #edges to reverse so the tournament becomes acyclic (oracle).
110     Try reversing subsets of edges in increasing size."""
111     edges = list(T.arc)
112     if is_acyclic(T):
113         return 0
114     for k in range(1, len(edges) + 1):
115         for combo in itertools.combinations(edges, k):
116             if is_acyclic(T.reverse(combo)):
117                 return k
118     return len(edges)
119
120
121 def fpt_bounded_search(T: Tournament, k: int) -> Optional[List[Edge]]:
122     """Return a set of <= k edges whose reversal makes T acyclic, or None.
123     Branch on a triangle: at least one of its 3 edges must be reversed.
124     Depth <= k, branching factor 3 => O(3^k * poly)."""
125     if is_acyclic(T):
126         return []
127     if k == 0:
128         return None
129     tri = find_triangle(T)
130     a, b, c = tri
131     for e in [(a, b), (b, c), (c, a)]:
132         sub = fpt_bounded_search(T.reverse([e]), k - 1)
133         if sub is not None:
134             return [e] + sub
135     return None
136
137
138 def kernelize(T: Tournament) -> Tournament:
139     """Remove vertices that lie in NO triangle (they never need a reversal).
140     A relabelled tournament on the surviving vertices is returned. (Part b's
141     kernel bound: if the answer is <= k, the surviving vertex count is O(k^2).)"""
142     in_triangle = set()
143     arc = T.arc
144     for a, b, c in itertools.permutations(range(T.n), 3):
145         if (a, b) in arc and (b, c) in arc and (c, a) in arc:
146             in_triangle.update([a, b, c])
147     keep = sorted(in_triangle)
148     relabel = {v: i for i, v in enumerate(keep)}
149     new_arcs = set()
150     for (u, v) in arc:
151         if u in relabel and v in relabel:
152             new_arcs.add((relabel[u], relabel[v]))
153     return Tournament(len(keep), new_arcs)
154

```